

Teoria de Grafos

Subgrafos e Cliques

Caminhos, Cadeias e Circuitos

Caminhamento em grafos

Prof. Dr. Marcos W. Rodrigues

marcos.rodrigues@

Roteiro de aula

- Subgrafos e Cliques
 - Subgrafos disjuntos de arestas
 - Subgrafos disjuntos de vértices e Clique
- Caminhos, Cadeias e Circuito
 - Sequência de arestas
 - Caminho aberto: Caminho simples
 - Caminho fechado: Circuito
- Caminhamento em grafos
 - Busca em profundidade (BFS)
 - Busca em largura (DFS)
 - Caminho mais curto / Menor caminho (Dijkstra)
 - Ordenação topológica
 - Conectividade e Atingibilidade

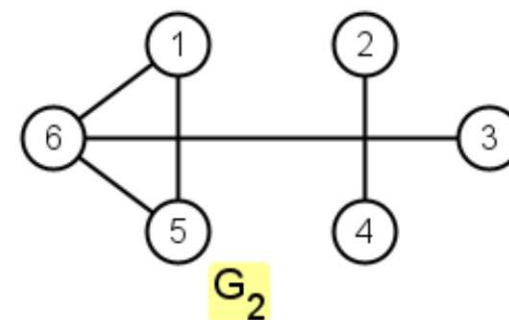
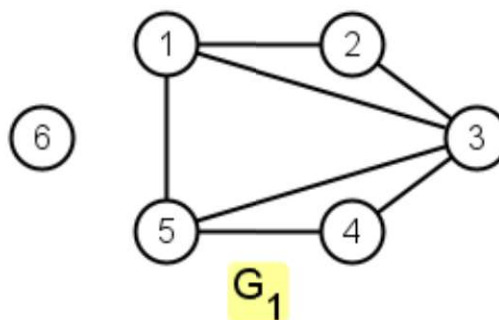
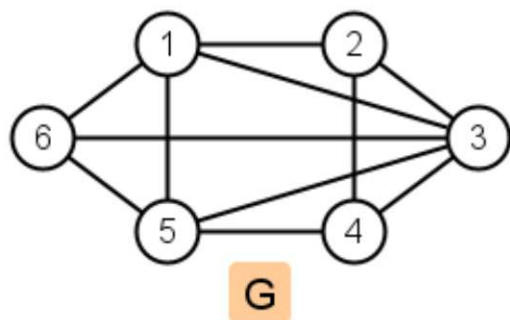
Subgrafos

Disjuntos de aristas

Disjuntos de vértices

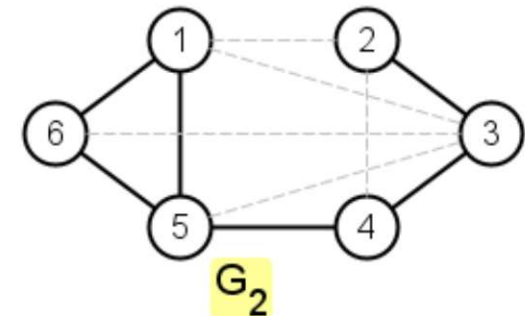
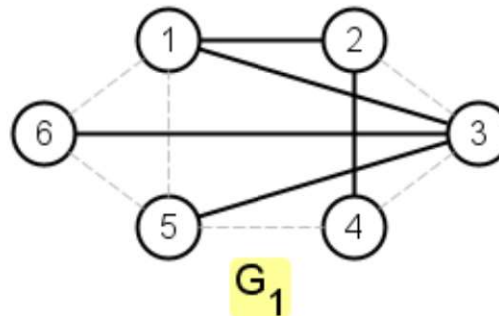
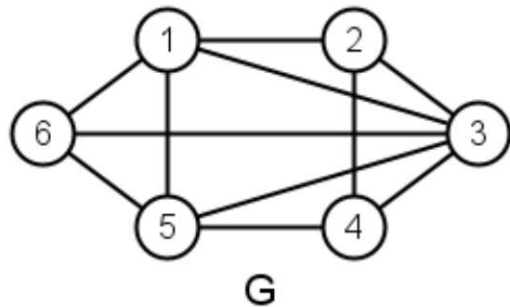
Subgrafos

- Um grafo $G_1 = (V_1, E_1)$ é dito ser um **subgrafo** de um grafo $G = (V, E)$, se **todos os vértices** e **todas as arestas** de $G_1 \subset G$, ou seja, $V_1 \subseteq V$ e $E_1 \subseteq E$
 - Todo grafo é subgrafo de si próprio
 - O subgrafo de um subgrafo de G é subgrafo de G
 - Um **vértice** de G é um **subgrafo** de G
 - Uma **aresta** de G (e os **vértices** das **extremidades**) é subgrafo de G



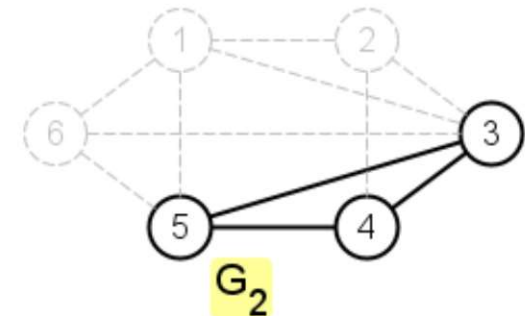
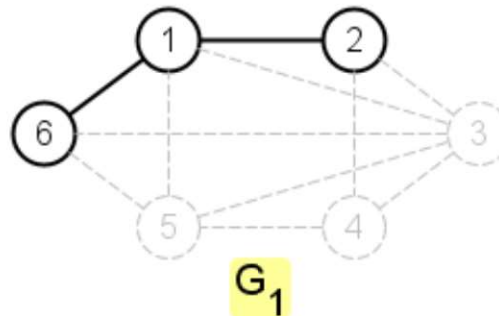
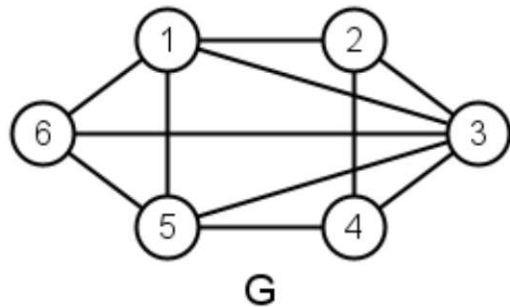
Subgrafos disjuntos de arestas

- Subgrafos disjuntos de arestas
 - Dois, ou mais, subgrafos G_1 e G_2 de um grafo G são **disjuntos de arestas** se G_1 e G_2 não tiverem arestas em comum
 - G_1 e G_2 podem ter **vértices em comum?**



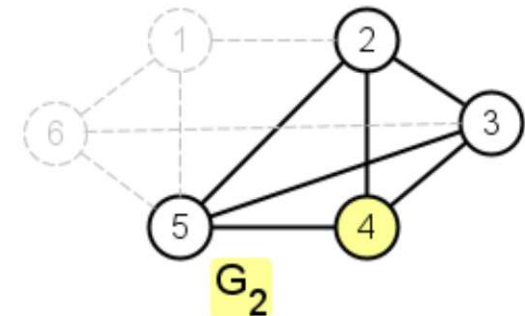
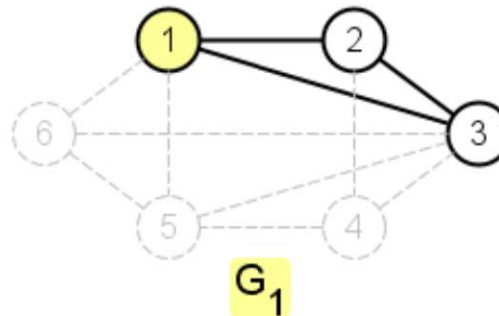
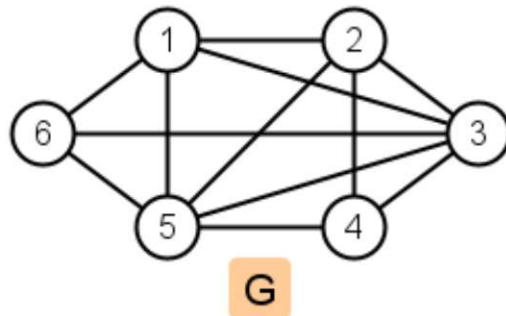
Subgrafos disjuntos de vértices

- Subgrafos disjuntos de vértices
 - Dois, ou mais, subgrafos G_1 e G_2 de um grafo G são **disjuntos de vértices** se G_1 e G_2 não tiverem vértices em comum
 - G_1 e G_2 podem ter arestas em comum?



O Clique em Subgrafos

- Um **Clique** de G é um **subgrafo induzido** G_1 e G_2 , de tal modo que, o subgrafo seja um grafo **completo** K_n
 - G_1 e G_2 podem ter arestas em comum?



Caminhos, Cadeias e Circuitos

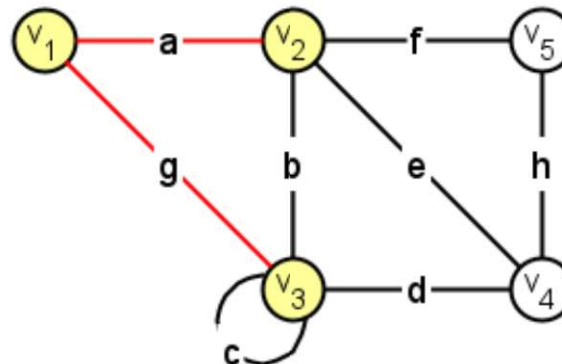
Sequência de arestas

Caminhos abertos: Caminho simples

Caminhos fechados: Circuitos

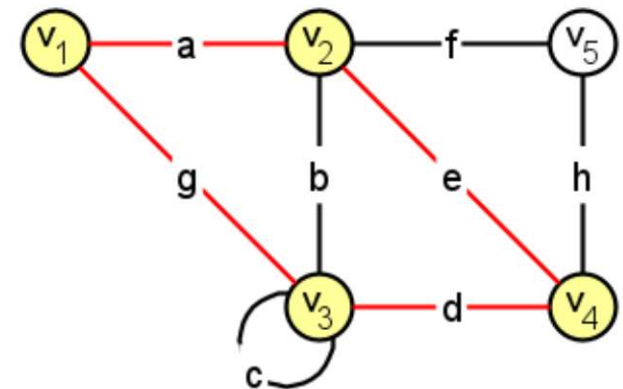
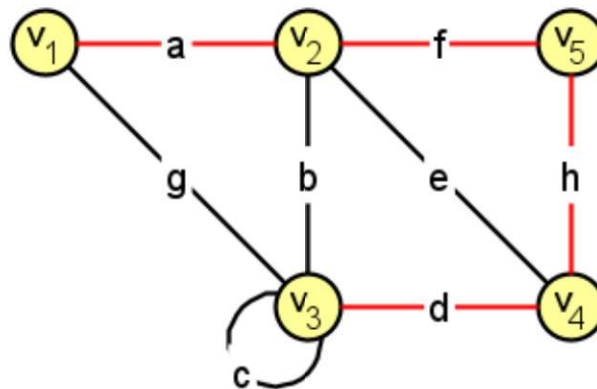
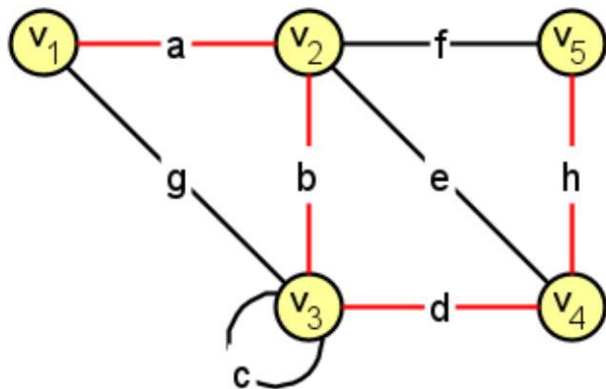
Sequencia de arestas

- Sequência de arestas
 - É uma sequência alternada de vértices e arestas, com início e fim em um vértice
- Cada aresta é incidente ao vértice que a precede e que a antecede
- Veja a sequência: $v_1 a v_2 a v_1 g v_3$



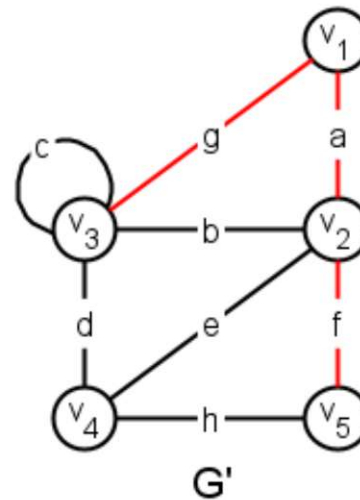
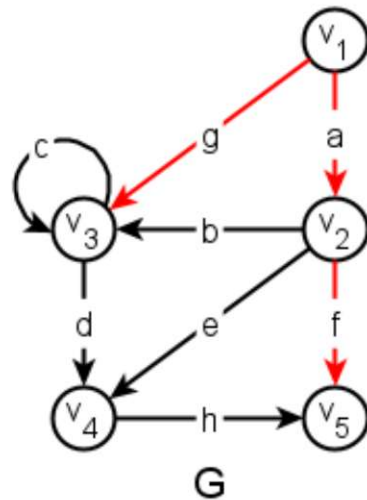
Caminho aberto e fechado

- Caminho é uma sequência de vértices e arestas, onde as arestas e vértices não se repetem, com exceção do circuito
 - Considere a seguinte sequência: $v_1 \bar{a} v_2 \bar{b} v_3 \bar{d} v_4 \bar{h} v_5$
 - No caminho aberto ou caminho simples não se repete arestas e nem vértices, por exemplo: $v_1 \bar{a} v_2 \bar{f} v_5 \bar{h} v_4 \bar{d} v_3$
 - No caminho fechado ou circuito não se repete arestas e nem vértices, exceção dos vértices inicial e final, $v_1 \bar{a} v_2 \bar{e} v_4 \bar{d} v_3 \bar{g} v_1$



Cadeias

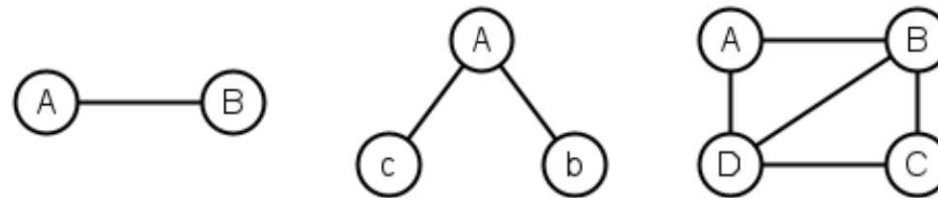
- Seja G um grafo dirigido e G' o grafo não-dirigido, o qual está associado ao grafo G
 - Uma **cadeia** em G é um **caminho** em G'



- As arestas $\bar{a}, \bar{f}$ e $\bar{g}$ é uma **cadeia** em G e um **caminho** de G'

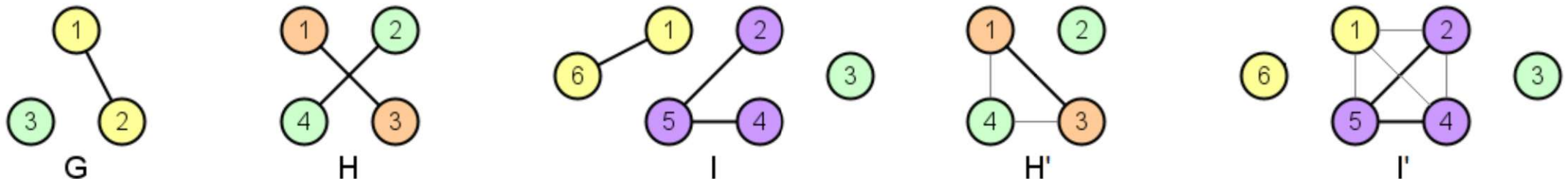
Relação entre Grafos e Componentes

- TEOREMA: Se um grafo G **conexo** possui **exatamente 2 vértices de grau ímpar**, então existe um **caminho (simples)** entre os vértices v_i e v_j



- TEOREMA: Um grafo G **simples** com n vértices e k componentes possui um número **máximo de arestas** $|E| = \frac{(n-k)(n-k+1)}{2}$

- O número **máximo de arestas** é **alcançável** ao obter um **subgrafo completo**, juntamente com **vértices isolados**, com mesmo **número de componentes** (H' e I')



- TEOREMA: O número **mínimo de arestas** de um grafo G **simples** com n vértices e k componentes é $n - k$, com mesmo **número de componentes**

Caminhamento em grafos

Busca em profundidade (BFS)

Busca em largura (DFS)

Caminho mais curto (Dijkstra)

Ordenação topológica

Caminhamento em grafos

- Busca em **profundidade** (DFS: *Depth-First Search*)
 - Realiza o **caminhamento** no grafo, **visitando todos os vértices** de G , buscando o **vértice mais profundo**, ou seja, o **próximo vértice** diretamente **conectado** e ainda **não visitado**
 - O DFS pode ser utilizado para **verificar** se um grafo **contém circuito**
- Busca em **largura** ou **amplitude** (BFS: *Breadth-First Search*)
 - **Expand** o **conjunto de vértices uniformemente**, de modo que são **visitados todos os vértices** de mesma “distância” (adjacentes) do **vértice verificado**, antes de visitar os vértices de **níveis posteriores**
- Tanto o **DFS** quanto o **BFS** podem ser usados para **identificar** o **número de componentes k** de um grafo G
 - Basta **verificar** se **existe** um **caminho** para **todos** os **pares de vértices**

Depth-First Search (Pseudo código)

BuscaProfundidade(int v_i)

for $w = 1, 2, \dots, n$ **do**
 visitado[w] = false
 antecessor[w] = null

end for

for $w \in V(G)$ **do**
 se !visitado[w]
 DFS(w)

end if

end for

end function

DFS(int w)

 visitado[w] = true

for $v \in G.\text{Adj}[w]$ **do**

if !visitado[v]

 antecessor[v] = w

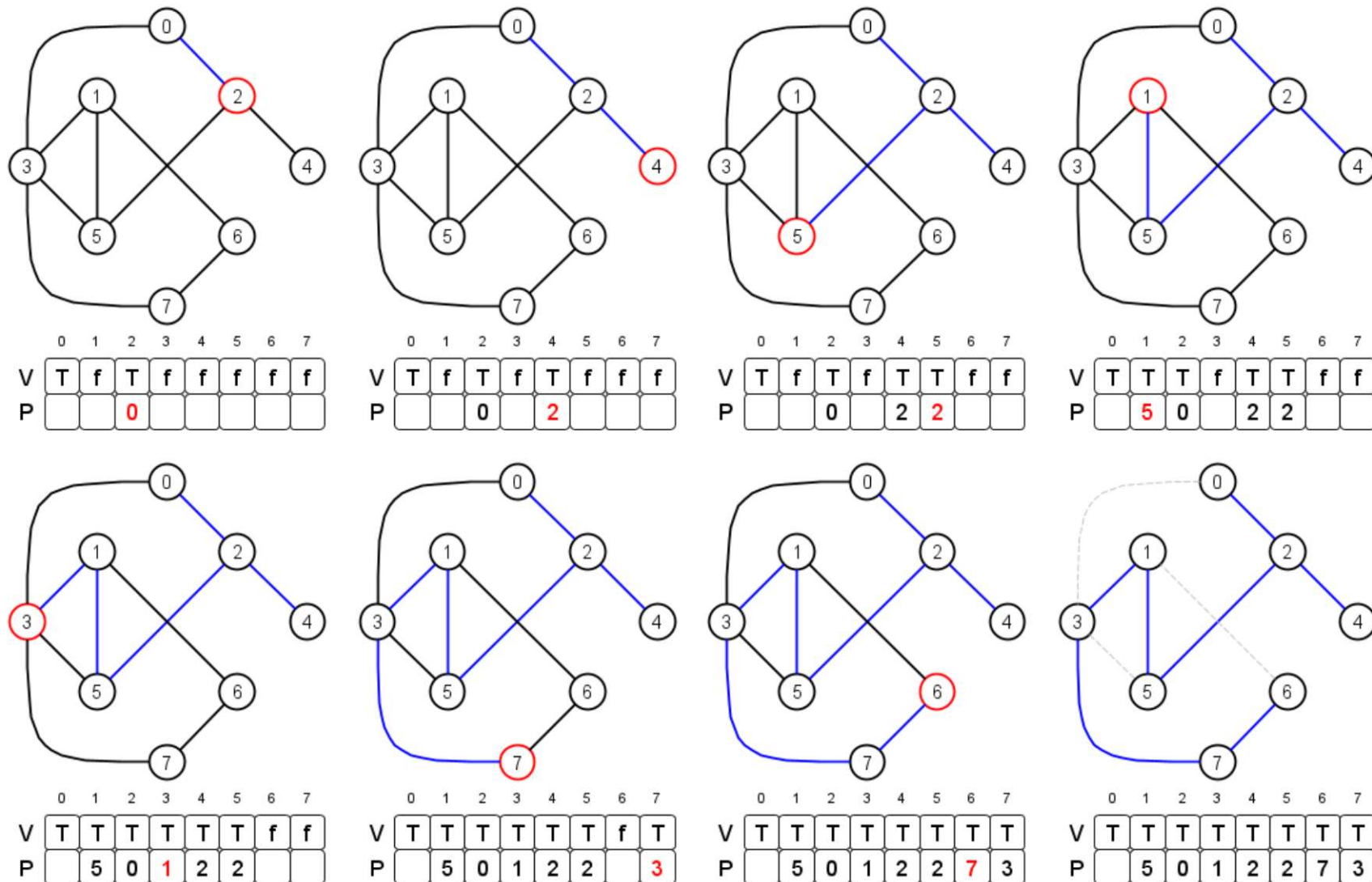
 DFS(v)

end if

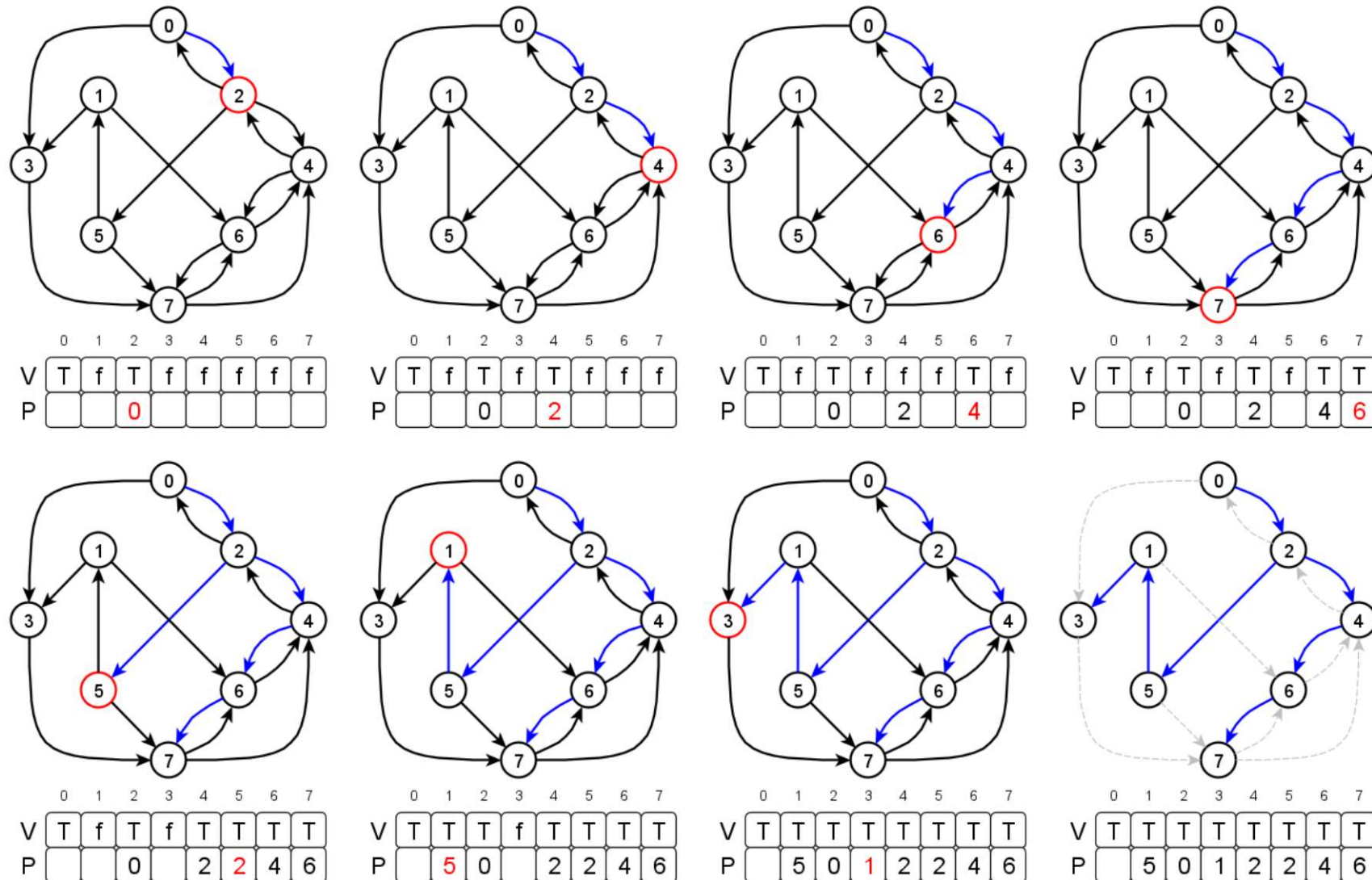
end for

end function

Busca em profundidade – Não direcionado



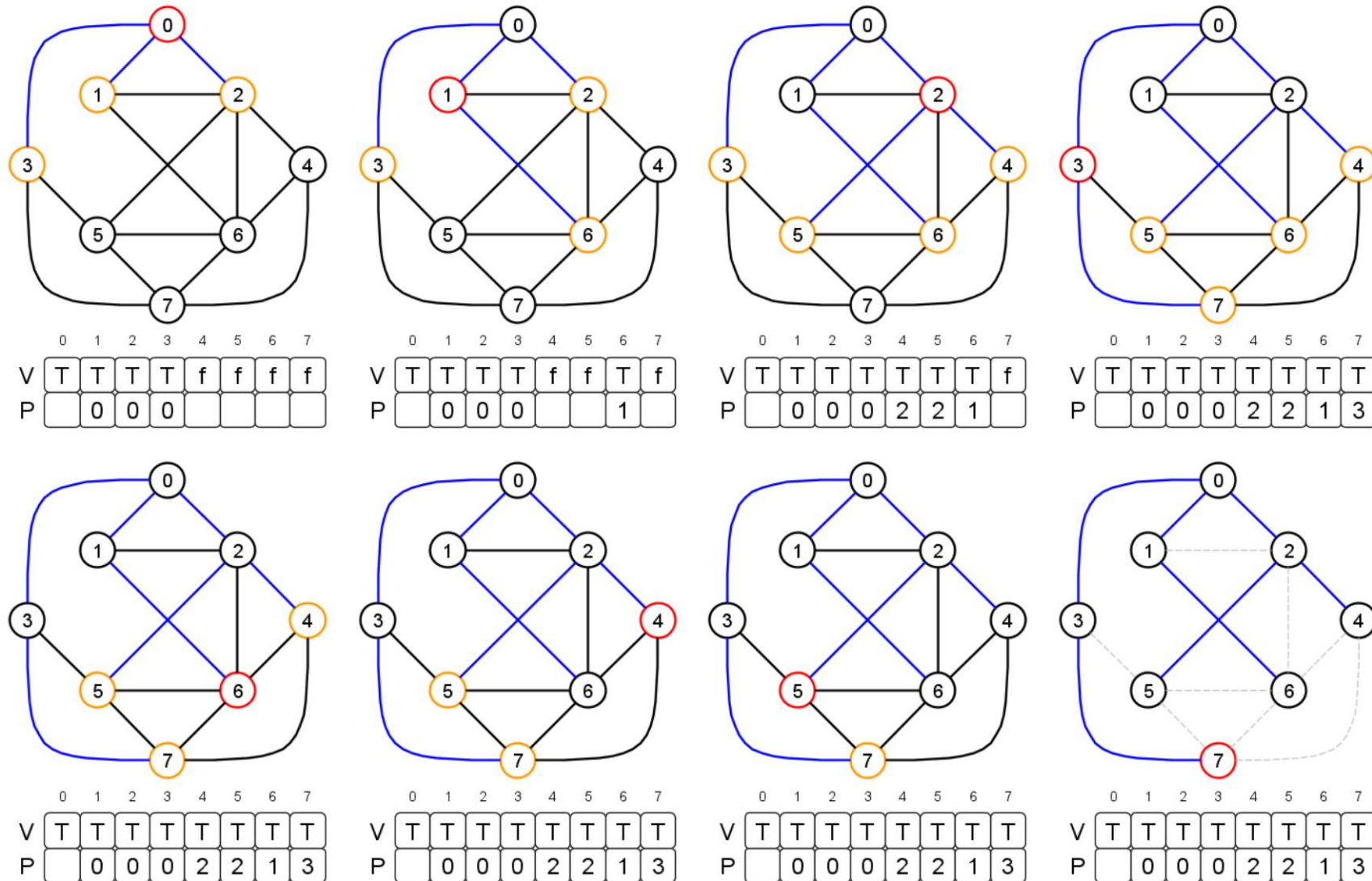
Busca em profundidade – Direcionado



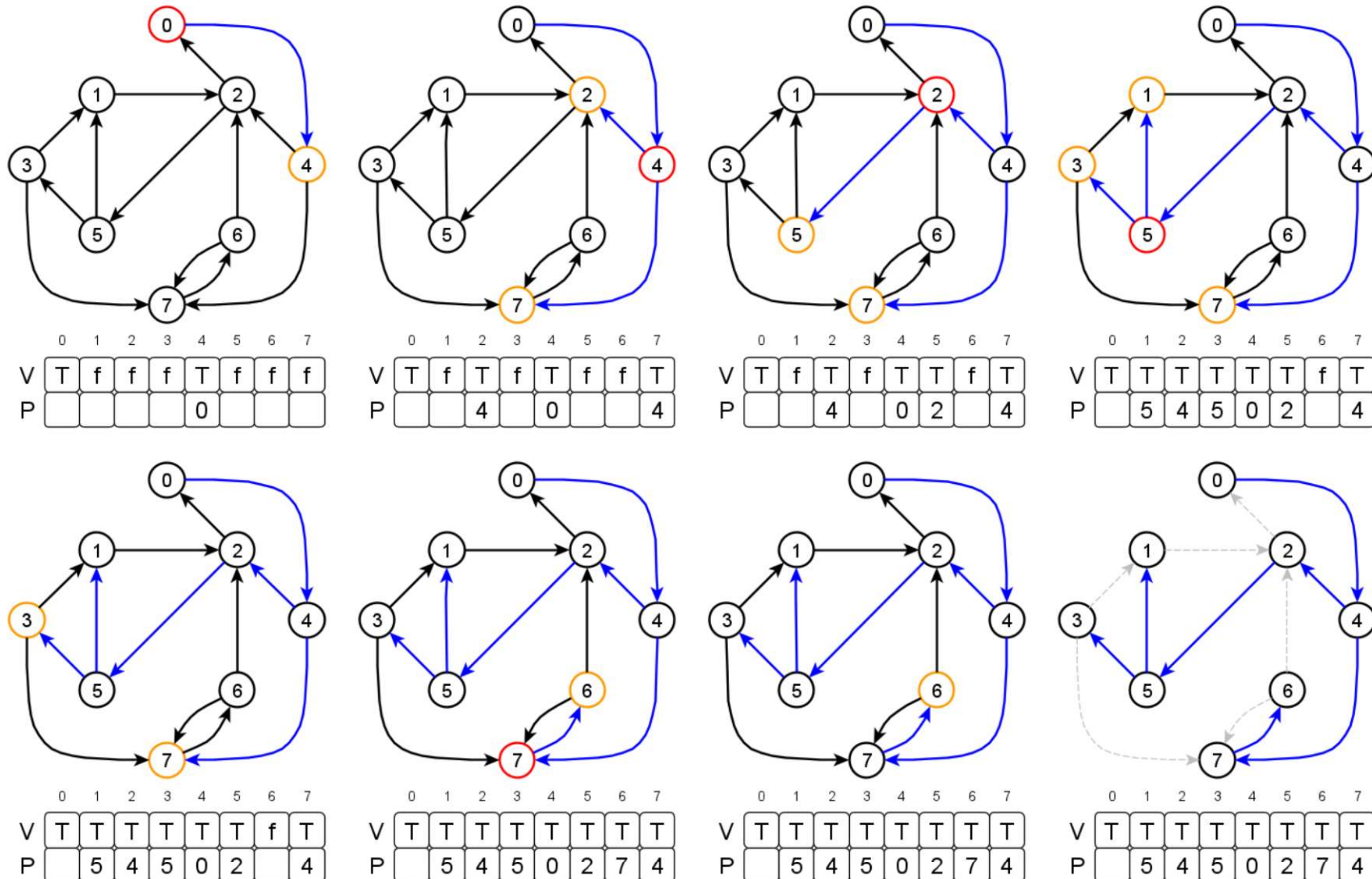
Breadth-First Search (Pseudo código)

```
BuscaAmplitude(int  $v_i$ )  
  visitado[v] = true  
  Fila.insere(v)  
  while !Fila.vazia() do  
    w = Fila.retira()  
    for z  $\in$  V(G) do  
      if E(z,w) e !visitado[z]  
        antecessor[z] = w  
        visitado[z] = true  
        Fila.insere(z)  
      end if  
    end for  
  end while  
end function
```

Busca em amplitude – Não direcionado



Busca em amplitude – Direcionado

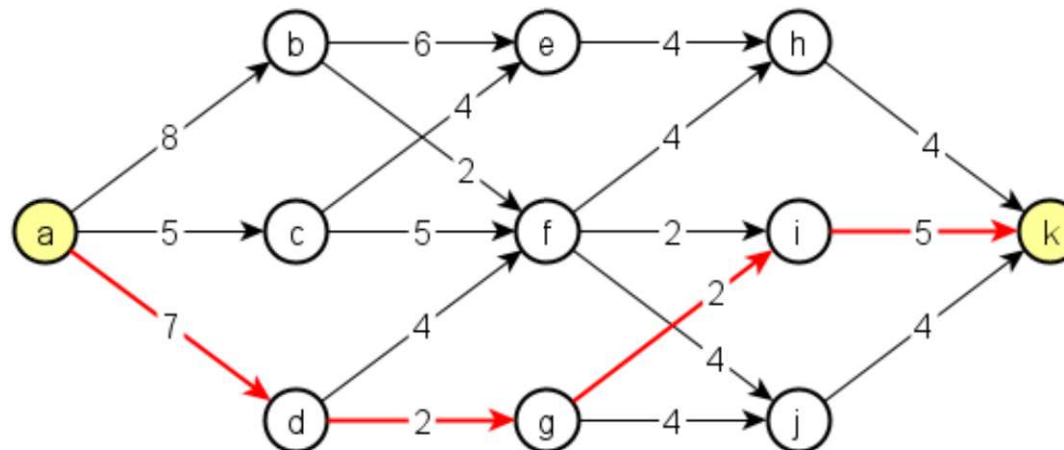


Algoritmo do menor caminho

- Seja o grafo $G = (V, E)$ orientado e com distância c_{ij} associada à aresta $(i, j) \in E$
- O objetivo é encontrar o caminho mais curto entre dois pares de vértices (v, u)
- O comprimento de um caminho é a soma dos comprimentos das arestas que compõem o caminho
 - O comprimento ou valor de uma aresta pode ser representado como: Custo; Distância; Consumo; Velocidade; Vazão, etc
- Exemplo de aplicação:
 - Dado um mapa rodoviário, determinar:
 - A rota mais curta entre dois pontos (cidades, bairros, esquinas, etc)
 - A rota com menor consumo de combustível ou a rota com menor valor de pedágio
 - Largura de banda de internet ou distribuição de sinal
 - Fluxo ou escoamento de carga/produtos

Algoritmo do menor caminho

- Encontre o **menor caminho** entre os vértices a e k
 - O **objetivo** é **construir uma estrada** entre **duas cidades**
 - O grafo abaixo representa os **percursos** possíveis e o **custo de construção** de cada **estrada**
 - Determine o **trajeto ótimo** com **custo mínimo** de construção, isto é, encontrar o **caminho mais curto** de a a k em relação aos custos



Algoritmo de Dijkstra

- Considere um grafo $G = (V, E)$ simples, cujas arestas possuem pesos não-negativos
- O algoritmo de Edsger Wybe Dijkstra (1975) resolve o problema do menor caminho entre pares de vértices
- Organização de dados:
 - $E(x, y)$: aresta de x para y
 - $W(x, y)$: peso da aresta de x para y
 - Incluídos: Vetor de vértices com menor caminho a partir de v_i já conhecidos
 - Borda: Vetor de vértices candidatos à próxima escolha
 - Antecessor[x]: Vértice antecessor ao vértice x de menor caminho
 - Custo[x]: Vetor de custo do caminho até alcançar o vértice x

Algoritmo de Dijkstra (Pseudo código)

DIJKSTRA(Grafo G , int v_i)

borda = $\{v_i\}$; incluídos = $\{\}$; antecessor[$v_i \sim v_f$] = -1

custo[$v_i \sim v_f$] = ∞ ; custo[v_i] = 0

while borda não vazia **do**

v = vértice da borda com o menor custo

 insira v em incluídos; retire v da borda

for todo $x \in V(G)$ tal que ($x \notin$ incluídos)

if $E(v, x)$ e ($\text{custo}[x] > \text{custo}[v] + w(v, x)$)

$\text{custo}[x] = \text{custo}[v] + w(v, x)$

 insira x na borda

 antecessor[x] = v

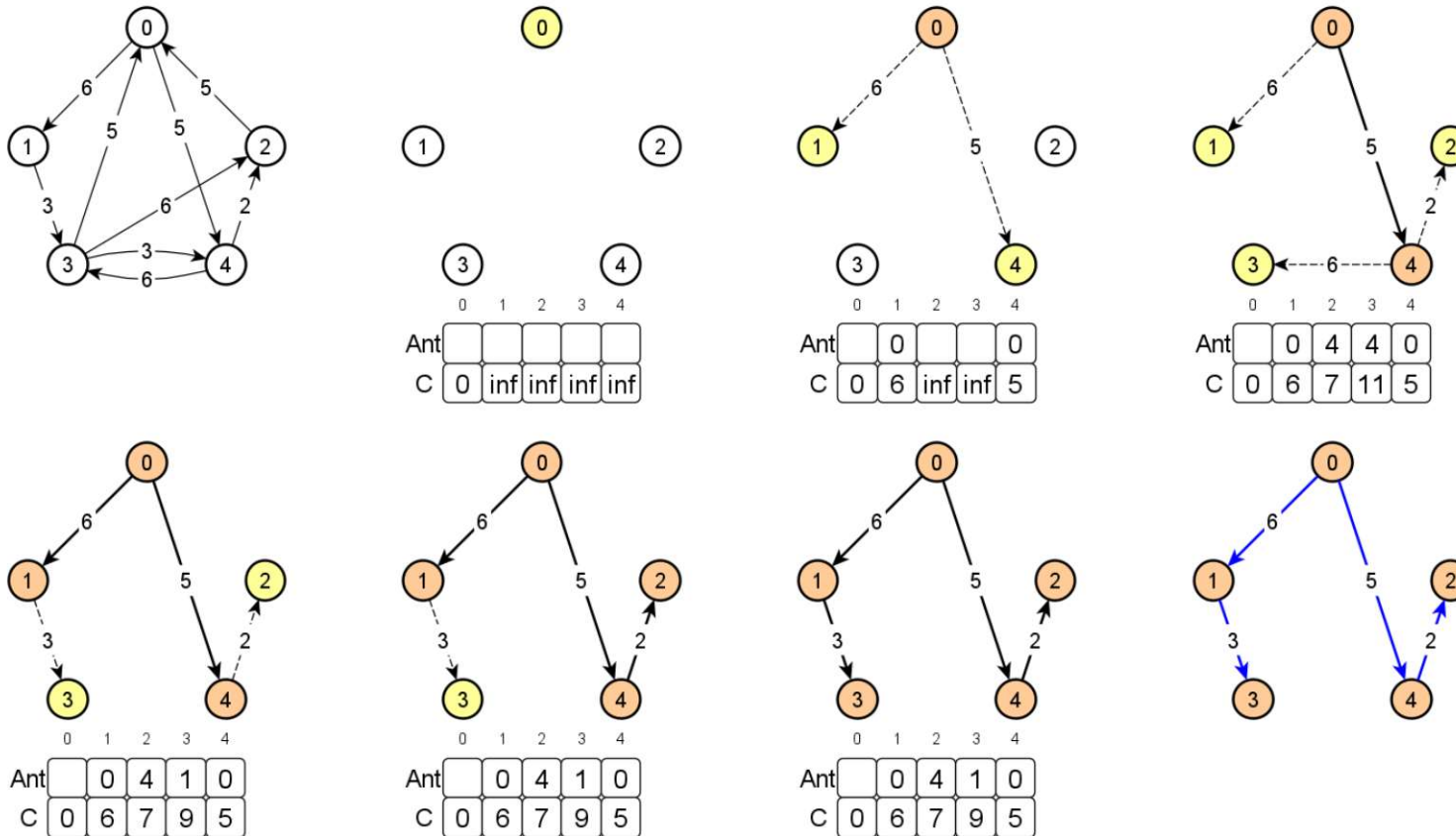
end if

end for

end while

end function

Algoritmo de Dijkstra



- Análise da eficiência (comportamento assintótico):
 - Grafos **densos** com **matrizes de adjacências**, tempo de execução é $O(n^2)$
 - Grafos **esparcos** com **listas de adjacências**, tempo de execução é $O(e \log n)$

Próxima aula...

- Atividades e compromissos:
 - Completar a modelagem dos problemas apresentados!
 - Implementar os algoritmos de Teoria de Grafos!

Até o nosso próxima encontro...